

DEPARTMENT OF COMPUTER SCIENCE

END OF TERM PROJECT REPORT

GRADEGUARDIAN

An Offline-First Academic Companion for Improving Student Performance

Submitted by

RICHARD YAWLUI

A web application project submitted in partial fulfilment of course requirements

July 2026

Declaration

I hereby declare that this project report, “GradeGuardian: An Offline-First Academic Companion for Improving Student Performance”, is the result of my own work carried out during the term, except where due acknowledgement has been made in the text. This work has not been submitted, either in whole or in part, for any other award.

Signature:

Name: Richard Yawlui

Date:

Abstract

Many students struggle to convert study effort into better grades because their academic information — course scores, deadlines, study time and revision material — is scattered across paper notes, chat apps and memory. Commercial solutions exist but typically require constant internet connectivity, paid subscriptions or cloud accounts, which are real barriers in environments where data costs are high and connectivity is unreliable.

This project presents GradeGuardian, a single-page web application that consolidates the tools a student needs to plan, track and improve academic performance: a course and grade manager with automatic GPA computation on the standard 4.0 scale, a Pomodoro-based study tracker with automatic session logging and streak motivation, a flashcard module implementing active recall with mastery tracking, a private document vault for notes and past questions, a deadline-aware task manager, and an analytics dashboard.

The system is deliberately offline-first: it is built with plain HTML5, CSS3 and vanilla JavaScript, stores structured data in the browser's `localStorage` and binary files in `IndexedDB`, and bundles all third-party assets locally, so it runs entirely without an internet connection and costs nothing to operate. Development followed an iterative methodology: an existing prototype was analysed, its defects were catalogued and fixed, and the system was redesigned and extended. Functional testing across all eight modules recorded a 100% pass rate on twenty test cases. The result demonstrates that a genuinely useful, private and free academic support tool can be delivered with core web technologies alone.

Keywords: academic performance, offline-first, GPA tracking, Pomodoro technique, active recall, flashcards, localStorage, IndexedDB, single-page application.

Table of Contents

Declaration.....	2
Abstract.....	3
Table of Contents.....	4
Chapter 1: Introduction	6
1.1 Background	6
1.2 Problem Statement.....	6
1.3 Aim and Objectives	6
1.4 Scope.....	6
1.5 Significance of the Project	7
Chapter 2: Review of Related Work.....	8
2.1 Study-Support Applications	8
2.2 The Learning Science Behind the Design	8
2.3 Gap Analysis	8
Chapter 3: Methodology and System Analysis	9
3.1 Development Methodology.....	9
3.2 Functional Requirements.....	9
3.3 Non-Functional Requirements.....	9
3.4 Technology Choices and Justification	10
3.5 System Architecture.....	10
3.6 Data Model	10
Chapter 4: System Design and Implementation	12
4.1 Visual Design	12
4.2 Course Management and Grading.....	13
4.3 Study Tracker	14
4.4 Flashcards (Active Recall).....	15
4.5 My Vault.....	16
4.6 Goals and Tasks.....	17
4.7 Settings and Data Management	18
4.8 Notable Implementation Details.....	19
Chapter 5: Testing and Results	20
5.1 Test Strategy	20
5.2 Defects Found in Version 1 and Their Resolution.....	20
5.3 Functional Test Cases.....	21
5.4 Results Summary.....	21
Chapter 6: Conclusion and Recommendations.....	23
6.1 Summary of Achievements	23

6.2 Limitations.....	23
6.3 Recommendations for Future Work	23
6.4 Conclusion.....	23
References	24
Appendix A: Project Structure	25
Appendix B: Grading Scale Used	26

(In Microsoft Word: right-click the table above and choose "Update Field" to refresh page numbers.)

Chapter 1: Introduction

1.1 Background

Academic success at university depends less on raw ability than on consistent habits: knowing where one stands in every course, distributing study time sensibly, revising with effective techniques, and never missing a deadline. Research on learning consistently shows that self-testing (active recall) and distributed practice are among the most effective study techniques, while simply re-reading notes is among the least effective (Dunlosky et al., 2013). Yet most students have no simple tool that brings grade awareness, time management and evidence-based revision together in one place.

GradeGuardian was conceived to close that gap. It is a lightweight web application a student can open in any browser — even without internet access — that acts as a personal academic companion throughout the semester.

1.2 Problem Statement

Students commonly (a) do not know their current GPA until results are published, and therefore cannot react in time; (b) study in unplanned, unmeasured bursts, making it impossible to see which courses are being neglected; (c) revise passively instead of testing themselves; and (d) lose track of assignment deadlines spread across notice boards and group chats. Existing applications that address parts of this problem require online accounts, continuous connectivity or subscription fees, and raise privacy concerns because academic records are stored on third-party servers.

1.3 Aim and Objectives

The aim of the project is to design and implement a free, fully offline, browser-based academic companion that helps students monitor and improve their performance.

The specific objectives are to:

- Implement course management with automatic GPA calculation on the standard 4.0 university grading scale;
- Provide a study tracker based on the Pomodoro technique that logs focus time automatically and motivates consistency through streaks and weekly statistics;
- Implement a flashcard module based on active recall, with per-card mastery tracking;
- Provide a private, on-device vault for lecture notes, images and PDF past questions;
- Provide deadline-aware task management with notifications for overdue and due-today items;
- Ensure the entire system operates with no internet connection, no account and no external database, with full data export/import for backup.

1.4 Scope

The system is a client-side single-page application for individual use. It covers course/grade tracking, study time tracking, flashcards, document storage, task management, analytics and personalisation (profile, themes, timer preferences). Multi-user collaboration, server synchronisation and mobile-native packaging are outside the current scope and are discussed as future work.

1.5 Significance of the Project

The project demonstrates that meaningful educational technology does not require cloud infrastructure. Because all data remains on the student's device, the tool is private by design, free to run, and usable in low-connectivity environments — characteristics particularly relevant to students in developing regions. Academically, the project exercises the full breadth of front-end engineering: state management, client-side persistence (Web Storage and IndexedDB), data visualisation, responsive design and accessibility.

Chapter 2: Review of Related Work

2.1 Study-Support Applications

Several categories of applications partially address the problem. Grade calculators (e.g. GPA calculator websites) compute averages but hold no persistent data and offer no behavioural support. Task managers such as Todoist or Google Tasks handle deadlines but know nothing about courses or grades. Focus timers such as Forest implement the Pomodoro technique but do not connect focus time to specific courses. Flashcard systems such as Anki and Quizlet implement active recall and spaced repetition powerfully, but Quizlet requires an account and connectivity, while Anki's interface presents a steep learning curve for casual users. No mainstream free tool integrates all four concerns, and nearly all require internet access.

2.2 The Learning Science Behind the Design

Three findings from cognitive science directly shaped GradeGuardian's features. First, the testing effect: practising retrieval of information produces substantially better long-term retention than re-studying it (Roediger and Karpicke, 2006). This motivates the flashcard module, where the student must attempt an answer before flipping the card, and honest self-assessment (“I knew it” / “Still learning”) drives mastery tracking. Second, distributed practice: spacing study across days beats massed “cramming” (Cepeda et al., 2006). The study streak and seven-day study chart make distribution — or the lack of it — immediately visible. Third, time-boxing: the Pomodoro technique (Cirillo, 2006) recommends fixed focus intervals separated by short breaks to sustain attention; the built-in timer implements configurable focus/break cycles.

2.3 Gap Analysis

The review shows a clear gap: an integrated, zero-cost, privacy-preserving, offline tool that combines grade awareness, measured study time, evidence-based revision and deadline management. GradeGuardian is designed to occupy exactly this gap.

Chapter 3: Methodology and System Analysis

3.1 Development Methodology

An iterative, prototype-driven methodology was used. Version 1 of the application was built first as a functional prototype. It was then critically evaluated: every defect and design weakness was catalogued (Section 5.2). Version 2 — the system described in this report — was produced by re-architecting the codebase around the findings: removing the cloud dependency, redesigning the data layer, fixing all recorded defects and extending the feature set. Each iteration ended with functional testing in the browser.

3.2 Functional Requirements

ID	Requirement
FR1	The user shall create, edit, delete and search courses with code, credit hours, instructor, semester and current score.
FR2	The system shall compute the letter grade, grade points and cumulative GPA (4.0 scale) automatically.
FR3	The user shall run a Pomodoro timer with configurable focus and break lengths.
FR4	The system shall log completed focus time against a selected course, automatically or on demand.
FR5	The system shall display study statistics: 7-day chart, weekly hours, total hours and day streak.
FR6	The user shall create flashcard decks and cards, and study them with flip-to-reveal and mastery tracking.
FR7	The user shall store text notes and upload images/PDF documents to a private vault with type, course and tag filters.
FR8	The user shall manage tasks with due dates and priorities, grouped as Overdue, Due Today, Upcoming and Completed.
FR9	The system shall surface overdue and due-today tasks as notifications with a badge count.
FR10	The user shall search courses, tasks, vault items and decks from a global search box.
FR11	The user shall export all data as a JSON backup and import it later.
FR12	The user shall personalise the profile, theme (light/dark) and weekly study goal.

3.3 Non-Functional Requirements

- Offline operation — the application must run with no internet connection; all assets (fonts, icons, chart library) are bundled locally.
- Privacy — all data remains on the user's device; no telemetry, accounts or external databases.
- Performance — pages must render instantly from local data; charts are redrawn only for the visible page.
- Usability — consistent visual language, empty states with guidance, keyboard support (Esc closes dialogs, Space flips flashcards).

- Responsiveness — usable from 360 px mobile widths to desktop, with a collapsing sidebar.
- Robustness — the timer must remain accurate when the browser throttles background tabs; corrupted stored data must not crash the application.

3.4 Technology Choices and Justification

Technology	Role	Justification
HTML5 / CSS3	Structure & presentation	Universally supported; CSS custom properties enable a complete light/dark theme system without a framework.
Vanilla JavaScript (ES6+)	Application logic	No build step, no dependencies to install; the entire app runs by opening index.html — ideal for an offline tool and for demonstrating core engineering skills.
localStorage	Structured data	Synchronous, simple key-value persistence for courses, tasks, sessions, decks, cards and settings.
IndexedDB	Binary file storage	localStorage is limited to ~5 MB of strings; IndexedDB stores image/PDF blobs natively and scales to hundreds of megabytes.
Chart.js 4 (bundled)	Data visualisation	Lightweight canvas charting for the grade, study-history and grade-distribution charts.
Font Awesome 6 + Poppins (bundled)	Iconography & typography	Professional visual finish while remaining fully offline.

3.5 System Architecture

GradeGuardian is a single-page application organised in three layers. The presentation layer is a set of eight page sections toggled by a hash-based router (`#dashboard`, `#courses`, ...), so browser back/forward and bookmarking work naturally. The logic layer contains feature modules (courses, grades, study timer, flashcards, vault, tasks, search/notifications, settings) plus shared services (theme engine, modal manager, toast system, chart factory). The persistence layer wraps localStorage behind typed accessors and wraps IndexedDB behind an asynchronous file-store API (`putFile`, `getFile`, `deleteFile`). A one-time migration routine upgrades data created by version 1 of the application, and a demo-data seeder populates a friendly first-run experience.

3.6 Data Model

Store	Location	Key fields
Course	localStorage: <code>gg_courses</code>	id, name, code, creditHours, instructor, semester, grade (0–100)
Task	localStorage: <code>gg_tasks</code>	id, title, course, dueDate, priority, completed
StudySession	localStorage: <code>gg_sessions</code>	id, courseId, courseName, minutes, date (ISO)
Deck	localStorage: <code>gg_decks</code>	id, name, course, desc, createdAt
Card	localStorage: <code>gg_cards</code>	id, deckId, front, back, streak (mastery ≥ 2)

Store	Location	Key fields
VaultItem	localStorage: gg_vault	id, kind (text file), title, type, course, tags, content / fileName, fileType, fileSize
File blob	IndexedDB: files	id (shared with VaultItem), blob
Settings	localStorage: gg_settings	name, email, theme, weeklyGoal, focusLen, breakLen

Chapter 4: System Design and Implementation

4.1 Visual Design

The interface follows a dashboard pattern: a fixed sidebar for navigation, a header with global search, notifications and a theme toggle, and a content area of cards. All colours are defined once as CSS custom properties; a data-theme attribute on the document root switches the entire palette between light and dark modes. The saved theme is applied by an inline script before first paint, eliminating any flash of the wrong theme. The Poppins typeface, soft shadows, rounded cards and a restrained indigo/pink accent palette give a modern, calm appearance appropriate for long study sessions.

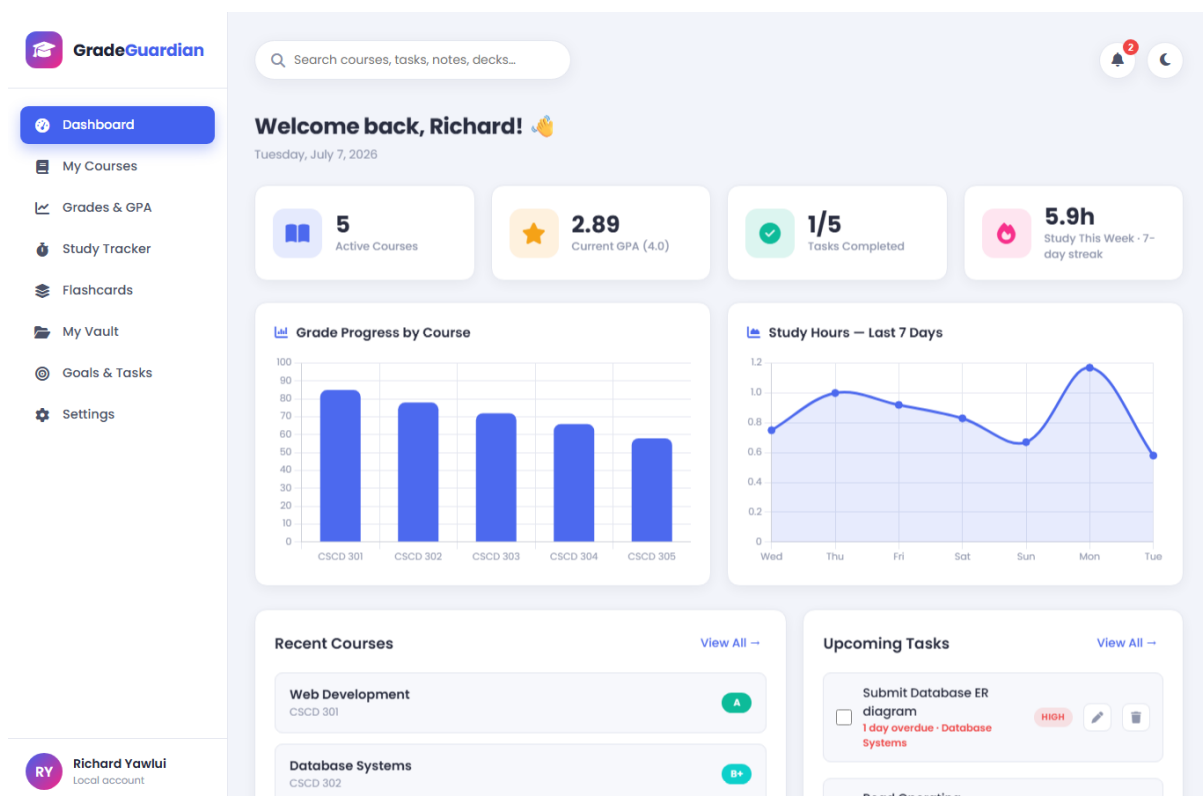


Figure 1: The dashboard — statistics, charts, recent courses, upcoming tasks and quick timer.

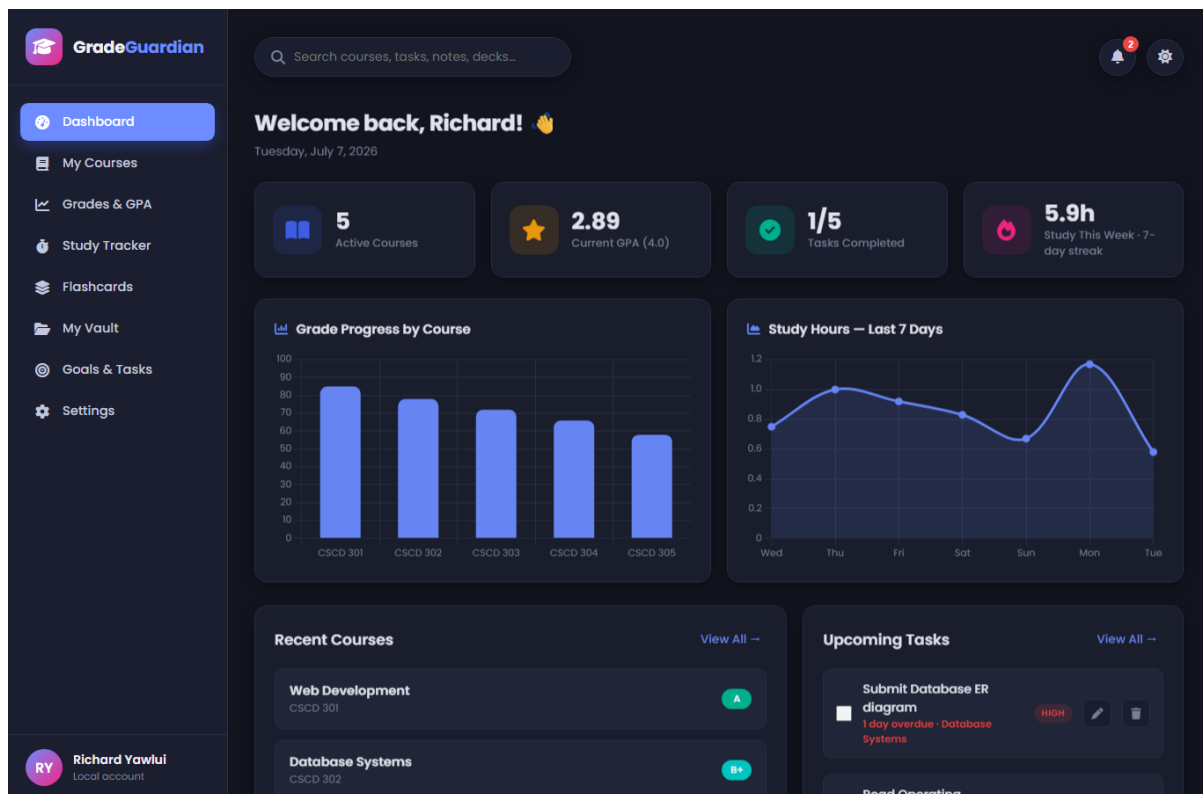


Figure 2: The same dashboard in dark mode. One attribute switch re-themes every component, including charts.

4.2 Course Management and Grading

Courses are managed through a modal form with live letter-grade preview: as the score is typed, the corresponding letter appears immediately. The course list supports text search, performance filtering (Strong / Average / At-Risk) and sorting by name or grade. A statistics strip summarises totals, average score, credit hours and the performance distribution.

Grading follows the standard scale used by Ghanaian universities: A (80–100, 4.0), B+ (75–79, 3.5), B (70–74, 3.0), C+ (65–69, 2.5), C (60–64, 2.0), D+ (55–59, 1.5), D (50–54, 1.0), F (below 50, 0.0). The cumulative GPA is the credit-hour-weighted mean of grade points: $GPA = \frac{\sum(\text{points} \times \text{credit hours})}{\sum(\text{credit hours})}$.

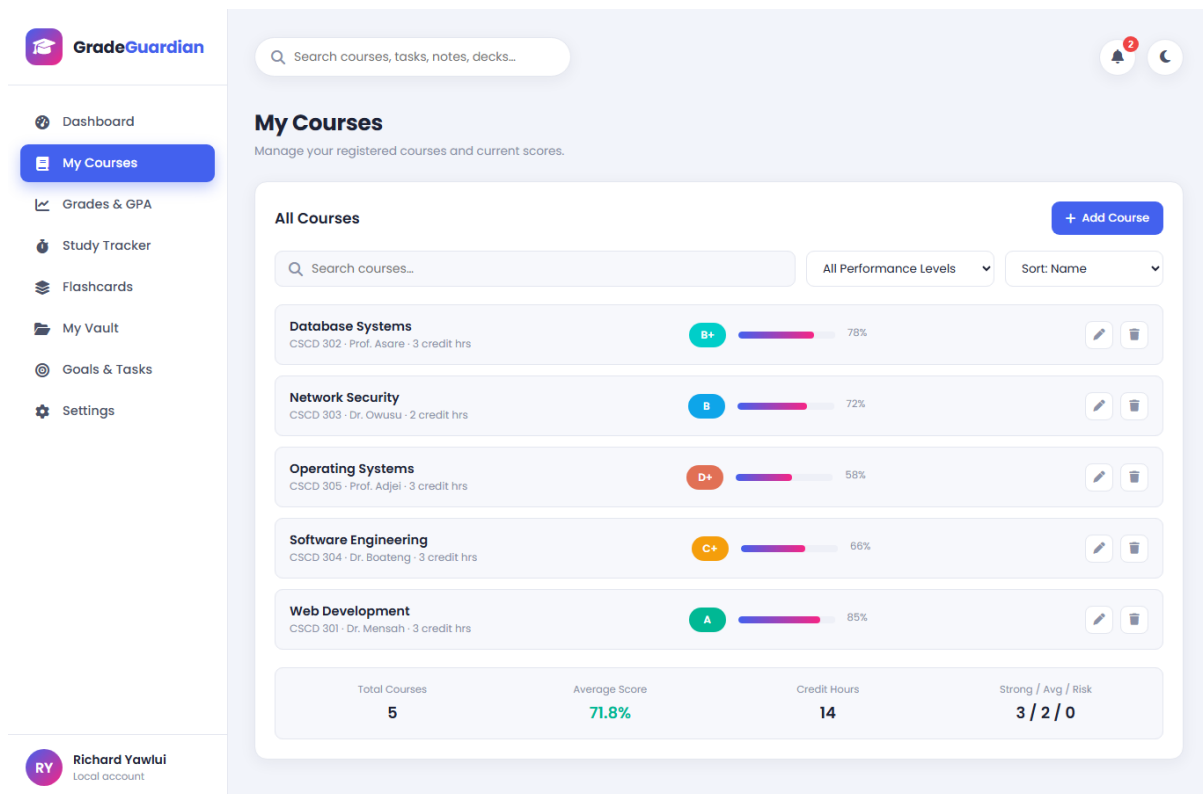


Figure 3: Course management with search, filters, sorting, per-course actions and summary statistics.



Figure 4: Grade analysis — GPA cards, letter-grade distribution chart, grading scale reference and per-course breakdown.

4.3 Study Tracker

The tracker implements the Pomodoro technique with configurable focus and break lengths. A key implementation detail is drift-free timing: browsers throttle `setInterval` callbacks in background tabs, so instead of counting ticks the timer records real timestamps and subtracts elapsed wall-clock time on every tick. Focus time accumulates in an “unlogged minutes” buffer; when a focus block completes it is logged automatically against the selected course, and the student can also log partial time manually. The page shows a seven-day history chart, total and weekly hours, the current day streak, and recent sessions with the ability to remove mistaken entries.

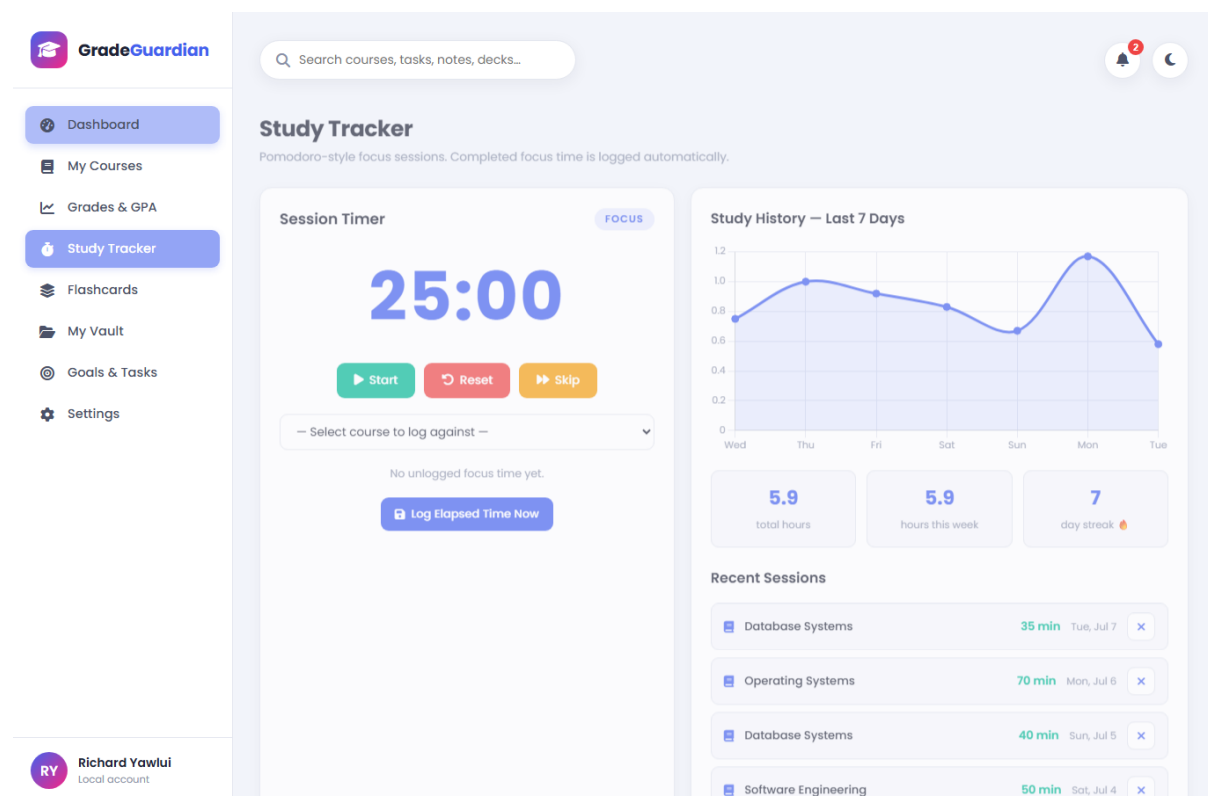


Figure 5: Study tracker — Pomodoro timer, automatic session logging, 7-day history, streak and recent sessions.

4.4 Flashcards (Active Recall)

Decks group cards by topic and optionally by course. In study mode the deck is shuffled with unmastered cards first, the question face is shown, and the student flips the card (click or Space bar) before honestly reporting whether they knew the answer. A correct streak of two or more marks a card as mastered; a miss resets it. Deck tiles show a mastery progress bar, giving a concrete revision target.

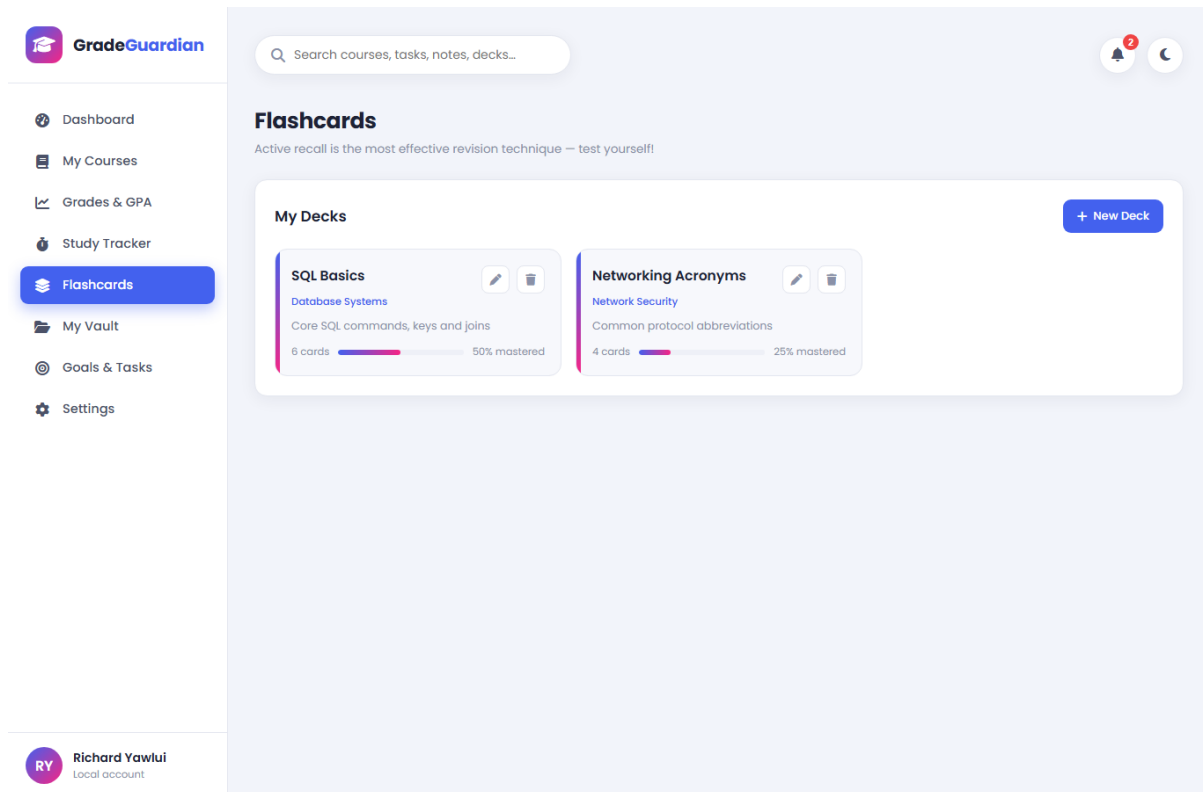


Figure 6: Flashcard decks with mastery progress. Study mode presents cards in a 3-D flip interface.

4.5 My Vault

The vault accepts written text notes and uploaded documents (images and PDFs, up to 10 MB each). Files are stored as blobs in IndexedDB, so they survive browser restarts yet never leave the device. Items carry a type (note, past question, other), a course and free-form tags; the grid can be filtered by any combination and searched by title or tag. Opening an item shows an inline image preview or a downloadable PDF entry.

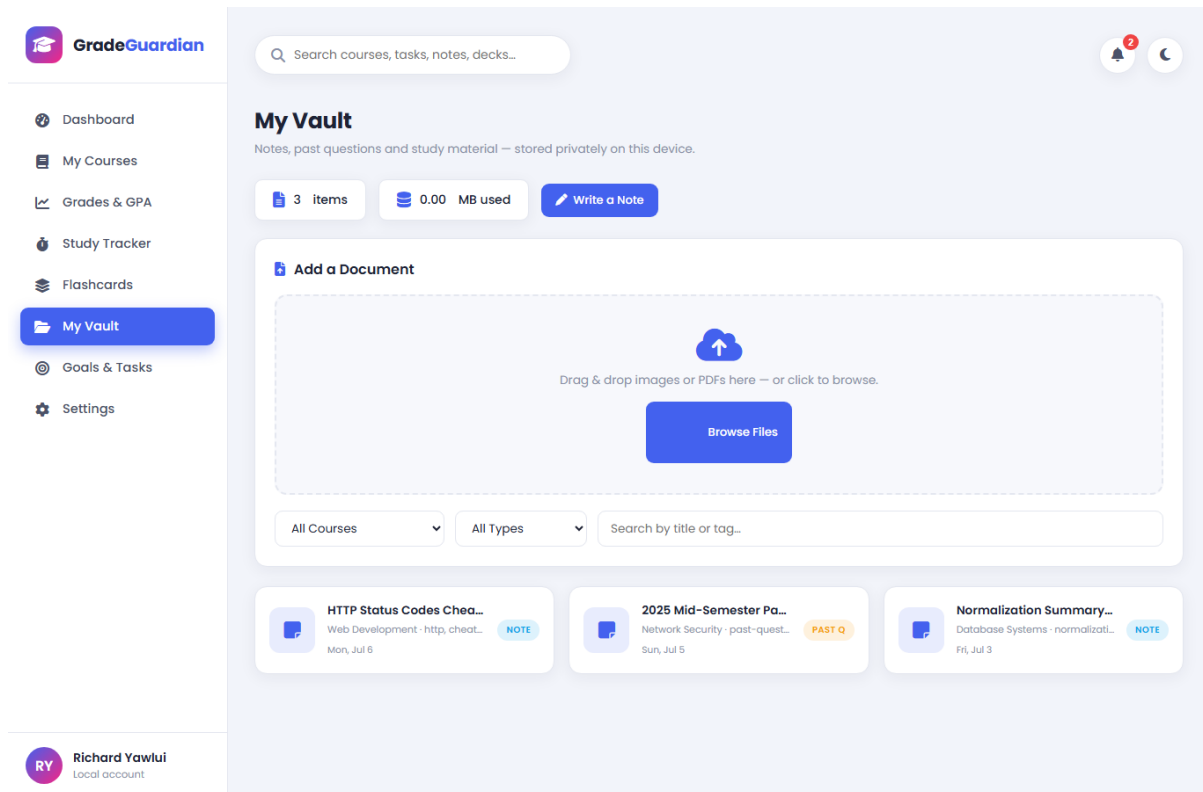


Figure 7: The vault — private, on-device storage for notes and past questions with filtering and tagging.

4.6 Goals and Tasks

Tasks carry a title, optional course, due date and priority. The list is grouped into Overdue, Due Today, Upcoming, No Due Date and Completed, with a completion progress bar. Overdue and due-today items also feed the notification bell in the header, whose badge shows the number of items needing attention.

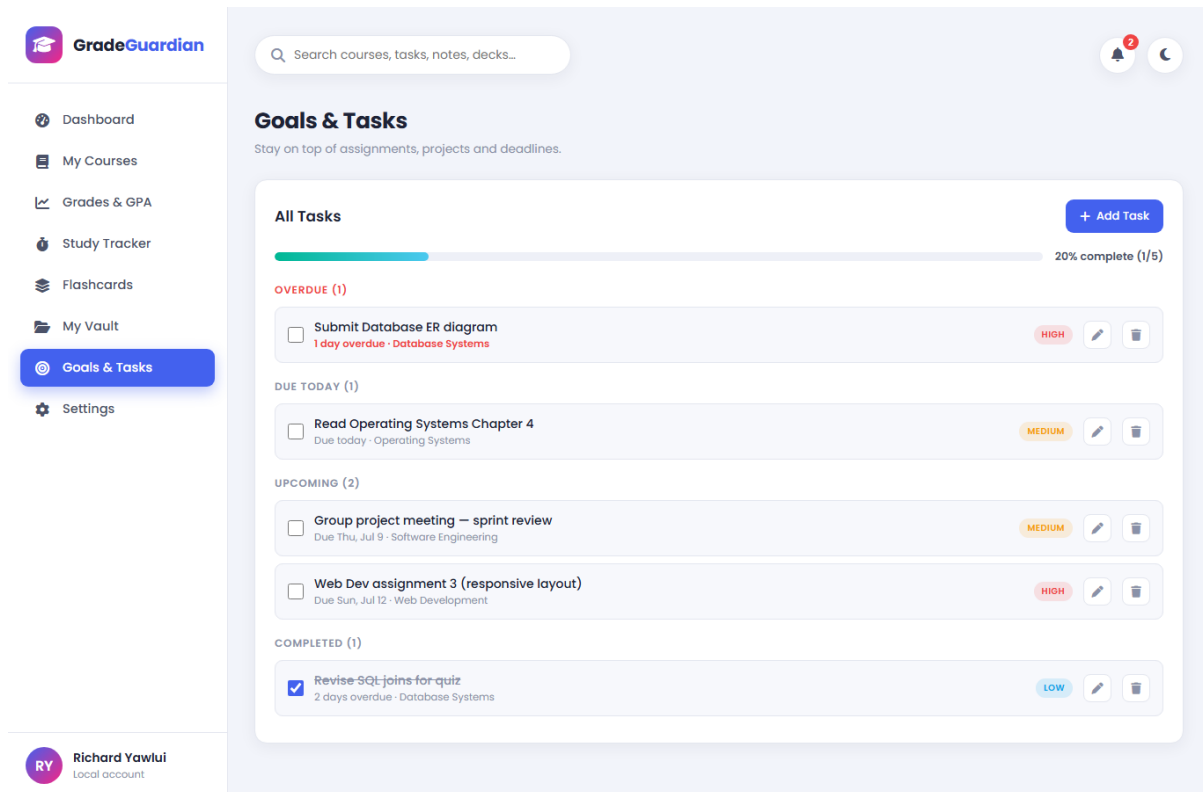


Figure 8: Task management with automatic grouping by urgency and a completion progress bar.

4.7 Settings and Data Management

Settings cover the profile (name and email drive the sidebar identity and dashboard greeting), preferences (focus/break lengths, weekly study goal, theme) and data management. The export function downloads a single JSON backup of all structured data; import validates and restores it. A demo-data loader and a full reset support demonstrations and fresh starts. Version 1 data, including from the old Firebase build, is migrated automatically on first launch.

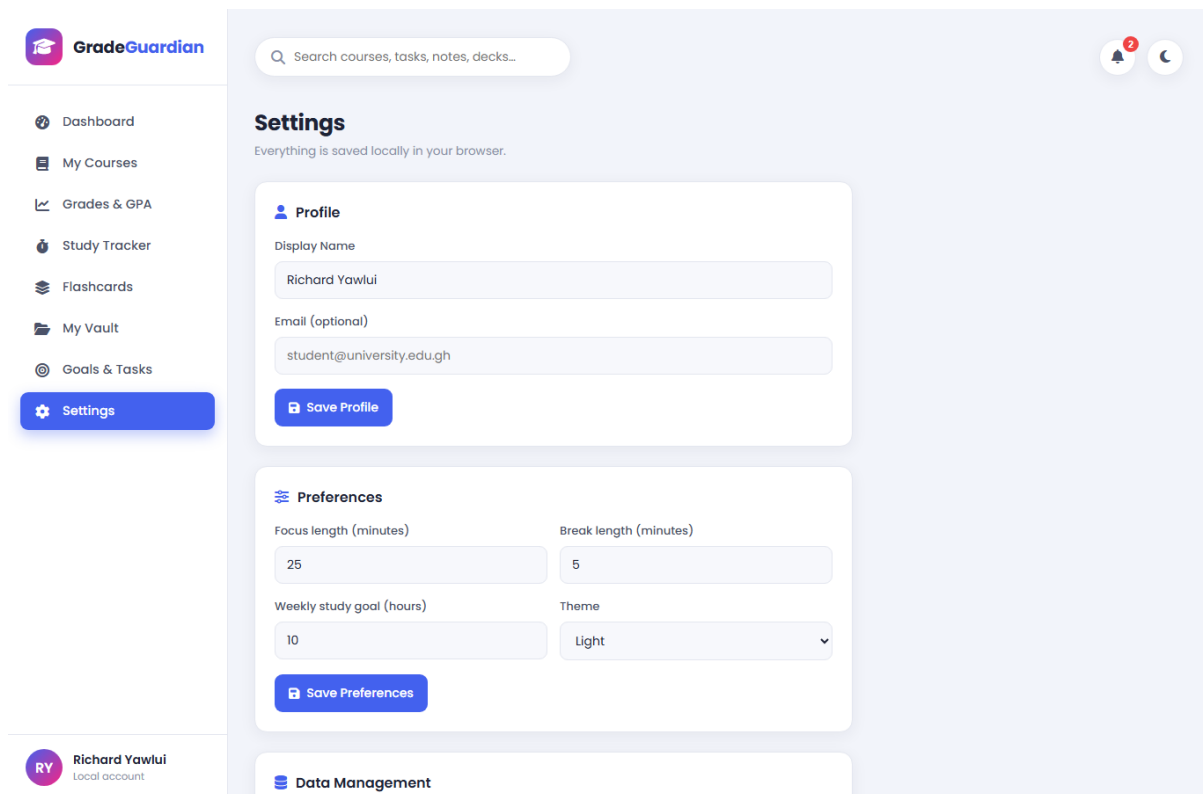


Figure 9: Settings — profile, preferences and local data management (backup, import, demo data, reset).

4.8 Notable Implementation Details

- Hash routing — each page has a URL (e.g. index.html#study), enabling deep links, bookmarks and browser navigation without a server.
- XSS-safe rendering — every user-entered string passes through an HTML-escaping function covering &, <, >, double and single quotes before insertion into the DOM.
- Chart theming — chart colours are resolved from the live CSS variables at draw time, so charts re-theme correctly when the user switches modes.
- Empty states — every list renders a friendly illustrated message with a call-to-action button when it has no data.
- Toast feedback — every action (save, delete, log, import...) confirms itself with a non-blocking toast notification.
- Custom confirm dialogs — destructive actions use a consistent in-app confirmation modal rather than the browser's native alert boxes.

Chapter 5: Testing and Results

5.1 Test Strategy

Functional testing was performed in the browser against each requirement, exercising the real user flows (forms, buttons, drag-and-drop) rather than internal functions where possible. The application was additionally driven by scripted DOM-level tests that simulated complete user journeys and asserted on both the visible interface and the persisted data. The browser console was monitored throughout; the final build produces no errors or warnings.

5.2 Defects Found in Version 1 and Their Resolution

A major activity of this iteration was the systematic repair of version 1. The table below records the most significant defects.

#	Defect in v1	Resolution in v2
1	File upload was impossible without a Firebase account and internet — uploads failed with an error in normal use.	Files are stored locally in IndexedDB; upload works fully offline with size/type validation.
2	Logging a study session with the timer untouched recorded 25 minutes of phantom study time.	The timer accumulates real elapsed focus time; logging requires at least one genuine minute.
3	Letter grades B+ and C+ had no matching badge styles, rendering unstyled; the CSS class “B+” is also invalid.	Letters map to safe CSS classes (b-plus, c-plus...) with a full colour scale for all eight letters.
4	GPA used a US-style 90/85/80 scale while letters used an 80/75/70 scale — the two disagreed for most scores.	One authoritative grading table drives letters, points and GPA consistently (Ghanaian 4.0 scale).
5	The “Save Changes” profile button had no event handler — profile edits were silently discarded.	Profile saving works and immediately updates the sidebar, avatar initials and dashboard greeting.
6	Global search and the notification bell were decorative and did nothing.	Search spans courses, tasks, vault and decks; the bell shows overdue/due-today items with a live badge.
7	Timer counted ticks with setInterval, drifting badly in throttled background tabs.	Timestamp-based timing keeps the timer accurate regardless of tab visibility.
8	HTML escaping ignored quotes, allowing malformed markup/attribute injection from user input.	Complete escaping of ampersands, angle brackets and both quote characters everywhere user text is rendered.
9	Profile avatar loaded from via.placeholder.com — a broken image when offline.	Initials avatar generated locally from the user's name.
10	The stylesheet contained ~250 duplicated lines (vault/auth styles pasted twice).	Stylesheet rewritten once around CSS variables; duplication removed.
11	Charts were re-created on every navigation, including for hidden pages, wasting memory.	Charts are drawn only for the visible page and destroyed before re-creation.
12	Semester options were hard-coded to 2024.	Semester options are generated from the current academic year.

5.3 Functional Test Cases

ID	Test case	Expected result	Result
T01	Open the app for the first time	Demo data seeds; dashboard renders with stats and charts	Pass
T02	Add a course with score 91	Course appears; preview shows letter A; GPA updates	Pass
T03	Edit a course score from 66 to 76	Badge changes C+ → B+; GPA and charts update	Pass
T04	Delete a course	Confirmation dialog; course removed; stats update	Pass
T05	Search and filter the course list	List narrows by text and performance band	Pass
T06	Start the study timer	Countdown runs; pause and reset behave correctly	Pass
T07	Complete a focus block with a course selected	Session logs automatically; toast confirms; break starts	Pass
T08	Log elapsed time manually with no course selected	Error toast "Select a course first"; nothing logged	Pass
T09	Attempt to log zero elapsed minutes	Error toast; no phantom session recorded	Pass
T10	Create a deck and add cards	Deck tile shows card count and 0% mastery	Pass
T11	Study a deck and answer honestly	Progress advances; streaks update; summary shows score	Pass
T12	Write, edit and delete a text note in the vault	Note persists across actions; delete asks for confirmation	Pass
T13	Upload an image and a PDF to the vault	Both stored in IndexedDB; image previews inline	Pass
T14	Upload an unsupported file type	File rejected with an explanatory toast	Pass
T15	Add tasks with past, today and future due dates	Tasks group into Overdue / Due Today / Upcoming	Pass
T16	Check the notification bell	Badge counts overdue + due-today; dropdown lists them	Pass
T17	Global search for a keyword	Matching courses, tasks, notes and decks listed; click navigates	Pass
T18	Toggle dark mode and reload	Theme persists; no flash of light theme; charts re-coloured	Pass
T19	Export a backup, reset all data, import the backup	All data restored exactly	Pass
T20	Reload while on #study with browser back/forward	Router restores the correct page each time	Pass

5.4 Results Summary

All twenty test cases passed (100%). The final build runs with zero browser-console errors and warnings, loads all assets locally, and was verified working with the network disabled. Responsive

behaviour was confirmed at desktop, tablet and mobile widths, with the sidebar collapsing to an icon rail below 768 px.

Chapter 6: Conclusion and Recommendations

6.1 Summary of Achievements

The project set out to build a free, offline, privacy-preserving academic companion, and delivered it: eight integrated modules, a consistent design system with light and dark themes, automatic GPA computation on the correct national grading scale, evidence-based study tools (Pomodoro time-boxing, streaks, active-recall flashcards), private document storage, and complete backup facilities — all in dependency-free vanilla JavaScript that runs by simply opening a file.

6.2 Limitations

Data lives in one browser profile on one device; clearing site data without an exported backup loses it. The JSON backup excludes uploaded binary files, which remain only in IndexedDB. Flashcards use simple mastery streaks rather than a full spaced-repetition scheduler. Grade tracking records one aggregate score per course rather than itemised assessments.

6.3 Recommendations for Future Work

- Assessment-level grades — record quizzes, assignments and exams with weights, and project the final grade.
- Spaced repetition — schedule flashcard reviews with an SM-2-style algorithm and daily review queues.
- Progressive Web App packaging — a service worker and manifest would enable installation and true offline caching.
- Optional encrypted sync — end-to-end encrypted backup to the student's own cloud drive, preserving privacy.
- Binary backup — include vault files in the export using a ZIP-based archive format.
- Reminders — local notifications for imminent deadlines via the Notifications API.

6.4 Conclusion

GradeGuardian shows that the browser alone is a complete application platform. By pairing sound learning science with careful front-end engineering, the project turns everyday web technology into a practical instrument for becoming a better student: one place to see the truth about your grades, your time and your revision — and to act on it.

References

Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3), 354–380.

Cirillo, F. (2006). *The Pomodoro Technique*. FC Garage.

Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., & Willingham, D. T. (2013). Improving students' learning with effective learning techniques. *Psychological Science in the Public Interest*, 14(1), 4–58.

Roediger, H. L., & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3), 249–255.

Mozilla Developer Network. (2025). Web Storage API; IndexedDB API. <https://developer.mozilla.org>

Chart.js contributors. (2024). Chart.js Documentation (v4). <https://www.chartjs.org>

Appendix A: Project Structure

Path	Purpose
index.html	Single-page application markup: sidebar, header, eight pages, six modals
style.css	Design system (CSS variables, light/dark themes) and all component styles
script.js	Application logic: storage layer, router, feature modules, charts, timer
vendor/chart.umd.min.js	Chart.js 4 (bundled for offline use)
vendor/fontawesome/	Font Awesome 6 icon set (bundled)
vendor/fonts/	Poppins typeface, four weights (bundled)
docs/	This report, the defense slides and page screenshots
backup_original/	Version 1 source, retained for reference
README.md	Quick-start guide

Appendix B: Grading Scale Used

Letter	Score range	Grade points	Interpretation
A	80 – 100	4.0	Excellent
B+	75 – 79	3.5	Very good
B	70 – 74	3.0	Good
C+	65 – 69	2.5	Fairly good
C	60 – 64	2.0	Average
D+	55 – 59	1.5	Below average
D	50 – 54	1.0	Marginal pass
F	0 – 49	0.0	Fail